Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Mise en œuvre

Microservices Maintenance & Driver

Etudiant :
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

3 avril 2026

Table des matières

1 Objectifs des nouveaux services **2**

2 Architecture de Persistance **2**
2.1 Isolation des instances 2
2.2 Initialisation via Kubernetes 2

3 Microservice Maintenance **2**
3.1 Entité MaintenanceRecord 2
3.2 Logique Métier et Alertes 3
3.2.1 Gestion des retards (Overdue) 3

4 Microservice Driver **3**
4.1 Entité Driver 4
4.2 Logique Métier et Conformité 4
4.2.1 Surveillance des Expirations (LicenseScheduler) 4

5 Intégration et Communication **5**
5.1 Exposition REST (APIs) 5
5.1.1 Endpoints Maintenance 5
5.1.2 Endpoints Driver 5
5.2 Synchronisation par Événements (Kafka) 5
5.2.1 Topic : flotte.maintenance.events 5
5.2.2 Topic : flotte.driver.events 5
5.3 Corrections Techniques Apportées 6
5.4 Exposition via le Gateway GraphQL 6
5.4.1 Implémentation du Gateway (Apollo Server) 6
5.4.2 Agrégation multi-services 6
5.4.3 Conteneurisation et Ingress 7

6 Architecture Événementielle (Saga Pattern) **7**
6.1 Transactions Compensatoires 7
6.2 Fiabilité des échanges (Outbox Pattern) 7
6.3 Flux de données 7
6.3.1 Flux Synchrones (Lecture et Agrégation) 8
6.3.2 Flux Asynchrones (Événements et Saga) 8

7 Tests automatisés et couverture de code **8**
7.1 Stratégie et Résultats 8

8 Déploiement Kubernetes et Helm **8**
8.1 Isolation de l'infrastructure (Helm) 8

1. Objectifs des nouveaux services

> **Objectif :** Implémenter les microservices de **Maintenance** (suivi technique) et **Driver** (gestion des conducteurs et permis) pour assurer la conformité et la disponibilité de la flotte.

Cette itération se concentre sur deux piliers essentiels :

— **Service Maintenance** : Responsable de la santé technique via la planification d'interventions et la détection de retards.

— **Service Driver** : Gestion administrative des conducteurs, incluant le suivi critique de la validité des permis de conduire.

2. Architecture de Persistance

Le système respecte strictement le pattern **Database per Service**. Chaque microservice métier dispose de son propre serveur PostgreSQL (StatefulSet) et de sa propre base de données, garantissant une isolation totale des données.

2.1. Isolation des instances

— **Véhicules** : Instance `postgres-service` (base `flotte_db`).

— **Maintenance** : Instance `postgres-maintenance-service` (base `maintenance_db`).

— **Driver** : Instance `postgres-driver-service` (base `driver_db`).

2.2. Initialisation via Kubernetes

L'initialisation des schémas est gérée par des **ConfigMaps** dédiés injectés dans les conteneurs au démarrage :

— `flotte-db-init` : Schémas pour les véhicules et la localisation.

— `maintenance-db-init` : Schéma `03-maintenance.sql`.

— `driver-db-init` : Schéma `02-driver.sql` et données initiales (*seeds*).

3. Microservice Maintenance

Le service Maintenance assure la traçabilité complète des interventions sur les véhicules.

3.1. Entité MaintenanceRecord

L'entité centrale `MaintenanceRecord` gère les cycles de vie des interventions via plusieurs énumérations (`Type`, `Status`, `Priority`).

```
@Entity
@Table(name = "maintenance_records")
public class MaintenanceRecord {
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private UUID vehicleId; // Lien logique vers le Service
        Vehicules

    @Enumerated(EnumType.STRING)
    private MaintenanceStatus status = MaintenanceStatus.SCHEDULED;

    private LocalDate scheduledDate;
    private Integer nextServiceKm; // Seuil pour les alertes
        preventives
}
```

Listing 1 – Entité MaintenanceRecord

3.2. Logique Métier et Alertes

Le service implémente des mécanismes proactifs pour assurer le suivi de la maintenance.

3.2.1. Gestion des retards (Overdue)

Une tâche planifiée (`@Scheduled`) s'exécute quotidiennement à 01h00 pour identifier les maintenances prévues dont la date est dépassée.

```
@Scheduled(cron = "0 0 1 * * ?")
@Transactional
public void checkOverdueMaintenance() {
    List<MaintenanceRecord> scheduledRecords = repository.
        findByStatus(MaintenanceStatus.SCHEDULED);
    LocalDate today = LocalDate.now();
    for (MaintenanceRecord record : scheduledRecords) {
        if (record.getScheduledDate().isBefore(today)) {
            record.setStatus(MaintenanceStatus.OVERDUE);
            repository.save(record);
            eventProducer.publishAlert(AlertEvent.overdue(
                record.getVehicleId(), "Maintenance en retard"));
        }
    }
}
```

Listing 2 – Tâche de vérification des retards

4. Microservice Driver

Le service Driver gère le cycle de vie des conducteurs et la conformité de leurs permis de conduire.

4.1. Entité Driver

Le service utilise une relation **Un-à-Plusieurs** entre l'entité `Driver` et `DriverLicense`. Il implémente également le *Soft Delete* via Hibernate.

```java
@Entity
@Table(name = "drivers")
@SQLDelete(sql = "UPDATE drivers SET deleted_at = CURRENT_TIMESTAMP
    WHERE id = ?")
@SQLRestriction("deleted_at IS NULL")
public class Driver {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String firstName;
    private String lastName;

    @Enumerated(EnumType.STRING)
    private DriverStatus status = DriverStatus.active;

    @OneToMany(mappedBy = "driver", cascade = CascadeType.ALL)
    private List<DriverLicense> licenses = new ArrayList<>();
}
```

Listing 3 – Entité Driver

4.2. Logique Métier et Conformité

Le service Driver assure la validité administrative des missions.

4.2.1. Surveillance des Expirations (LicenseScheduler)

Une tâche planifiée vérifie chaque nuit la validité des permis enregistrés pour alerter les gestionnaires avant l'expiration effective.

```java
@Scheduled(cron = "0 0 0 * * *")
@Transactional(readOnly = true)
public void checkLicenseExpirations() {
    // Detection des permis expirant dans 30 jours
    LocalDate in30Days = LocalDate.now().plusDays(30);
    licenseRepository.findByExpiryDate(in30Days).forEach(license ->
        {
        eventProducer.sendLicenseExpiringEvent(license);
    });

    // Detection des permis expires aujourd'hui
    LocalDate today = LocalDate.now();
    licenseRepository.findByExpiryDate(today).forEach(license -> {
        eventProducer.sendLicenseExpiredEvent(license);
    });
}
```

Listing 4 – Vérification proactive des permis

5. Intégration et Communication

Les services **Maintenance** et **Driver** sont pleinement intégrés à l'écosystème via des APIs REST (normalisées en `snake_case`) et des flux d'événements Kafka.

5.1. Exposition REST (APIs)

5.1.1. Endpoints Maintenance

Méthode	Endpoint	Description
POST	`/api/maintenance`	Planification initiale d'une fiche.
GET	`/api/maintenance`	**Nouveauté :** Liste globale filtrable par véhicule, statut ou priorité.
GET	`/api/maintenance/vehicle/{id}`	Historique complet d'un véhicule.
PUT	`/api/maintenance/{id}`	Mise à jour générale de la fiche.
PATCH	`/api/maintenance/{id}/status`	Clôture technique (coût, notes, etc.).

5.1.2. Endpoints Driver

Méthode	Endpoint	Description
GET	`/drivers`	Liste des conducteurs (filtrable par statut).
POST	`/drivers`	Création d'un nouveau profil conducteur.
PATCH	`/drivers/{id}/status`	Mise à jour du statut (`active`, `on_tour`, etc.).
POST	`/drivers/{id}/licenses`	Ajout d'un permis de conduire au profil.

5.2. Synchronisation par Événements (Kafka)

Le bus d'événements assure la cohérence entre les services sans couplage direct.

5.2.1. Topic : flotte.maintenance.events

Le service **Véhicules** consomme ces messages pour ajuster l'état opérationnel :

- `MAINTENANCE_STARTED` → Statut véhicule : `in_maintenance`.
- `MAINTENANCE_COMPLETED/CANCELLED` → Statut véhicule : `available`.

5.2.2. Topic : flotte.driver.events

Ce topic notifie les créations et mises à jour de conducteurs, ainsi que les alertes d'expiration de permis détectées par le `LicenseScheduler`.

```
{
  "eventId": "625c87b9-...",
  "eventType": "MAINTENANCE_COMPLETED",
  "timestamp": 1775220800.293,
```

```
  "payload": {
    "record_id": "7e3638d7-...",
    "vehicle_id": "39e75ab3-...",
    "cost_eur": 250.50
  },
  "metadata": { "triggered_by": "maintenance-service" }
}
```

Listing 5 – Format d'enveloppe unifie (KafkaEventEnvelope)

5.3. Corrections Techniques Apportées

L'implémentation a nécessité la résolution de plusieurs défis techniques majeurs rencontrés lors des tests d'intégration :

— **Routage et redirection (405 Method Not Allowed)** : Identification et correction d'une erreur 405 lors des requêtes `PUT`. Le problème provenait d'un conflit de port entre l'Ingress et le service Kubernetes, ainsi que d'une gestion ambiguë des slashes finaux dans les contrôleurs Spring Boot.

— **Désérialisation Kafka (ClassCastException)** : Résolution d'une erreur où les événements reçus étaient interprétés comme des `LinkedHashMap` au lieu des types Records attendus. La solution a consisté à configurer explicitement le `spring.json.type.mapping` dans les propriétés Kafka.

— **Cohérence des Schémas GraphQL** : Correction du crash du Gateway au démarrage dû à l'absence des types de pagination (`DriverPage`, `AlertPage`) dans les fichiers de définition du schéma.

— **Contraintes de base de données** : Gestion des erreurs 500 liées à la contrainte `chk_completed_date` (vérifiant que la fin d'une maintenance ne précède pas sa date prévue).

5.4. Exposition via le Gateway GraphQL

L'ensemble des services (Véhicule, Driver, Maintenance) est fédéré par une passerelle GraphQL implémentée avec Apollo Server.

5.4.1. Implémentation du Gateway (Apollo Server)

Une application Node.js dédiée centralise et expose le schéma GraphQL. Les requêtes sont résolues via un `Context` instanciant des clients REST (Axios) pour chaque microservice :

— **VehicleClient** : requêtes vers `http://vehicle-service:8080`

— **DriverClient** : requêtes vers `http://driver-service:8080`

— **MaintenanceClient** : requêtes vers `http://maintenance-service:8082`

5.4.2. Agrégation multi-services

L'API Gateway permet d'agréger les données de façon transparente pour le client :

— **Maintenance ↔ Conducteur** : Résolution du technicien.

— **Véhicule** ↔ **Maintenance** : Affichage de l'historique technique depuis la fiche véhicule.
— **Véhicule** ↔ **Conducteur** : Détail de l'assignation courante d'un véhicule vers le profil du conducteur (et ses permis).

5.4.3. Conteneurisation et Ingress

Le Gateway est buildé dans une image Docker `graphql-gateway:latest` et déployé sur le cluster Kubernetes (port 4000). La configuration de l'**Ingress** route désormais le trafic de `/graphql` vers cette passerelle.

6. Architecture Événementielle (Saga Pattern)

Le système implémente le pattern Saga par **Chorégraphie**, assurant la cohérence éventuelle entre les services distribués.

6.1. Transactions Compensatoires

Pour garantir l'intégrité des données, un mécanisme de compensation a été mis en œuvre sur le flux de maintenance :

1. **T1 (Action)** : Le service Maintenance crée ou démarre une intervention (`MAINTENANCE_STARTED`).
2. **T2 (Réaction)** : Le service Véhicule tente de passer le véhicule en `in_maintenance`.
3. **C1 (Compensation)** : En cas d'échec de T2, le service Véhicule publie un événement `MAINTENANCE_REJECTED`. Le service Maintenance consomme cet événement et **annule** automatiquement la fiche d'intervention (`cancelRecord`) pour restaurer l'état initial.

6.2. Fiabilité des échanges (Outbox Pattern)

Une réflexion a été menée sur la fiabilité de la "double écriture" (Base de données + Kafka). Bien que l'**Outbox Pattern** soit le standard industriel pour garantir l'atomicité parfaite entre la persistance et la publication d'événements, il n'a pas été retenu pour cette itération.

Compte tenu de la taille du projet et de la complexité opérationnelle qu'impliquerait la mise en place d'un outil comme *Debezium* ou d'un worker de relayage, nous avons privilégié le **Direct Publishing** au sein de transactions Spring `@Transactional`. Cette approche, combinée aux transactions compensatoires du pattern Saga, offre un excellent compromis entre robustesse et maintenabilité pour notre cas d'usage.

6.3. Flux de données

L'architecture repose sur deux modes de communication complémentaires pour assurer la fluidité et la cohérence des données.

6.3.1. Flux Synchrones (Lecture et Agrégation)

Les requêtes de consultation transitent par le **Gateway GraphQL**. Ce dernier joue le rôle d'orchestrateur :

- Il reçoit une requête unique (ex : fiche véhicule avec historique et conducteur).
- Il appelle en parallèle les services REST concernés via leurs APIs respectives.
- Il assemble les réponses JSON pour renvoyer un objet structuré au client.

6.3.2. Flux Asynchrones (Événements et Saga)

Les changements d'état critiques utilisent le bus de messages **Kafka** pour garantir le découplage :

- **Cycle de vie Maintenance** : `Maintenance` → `Kafka` → `Véhicule` (Mise à jour automatique de la disponibilité).
- **Mise à jour Kilométrique** : `Véhicule` → `Kafka` → `Maintenance` (Calcul des prochaines échéances et alertes).
- **Assignation Chauffeur** : `Véhicule` → `Kafka` → `Driver` (Passage du chauffeur en statut `on_tour`).

7. Tests automatisés et couverture de code

> **Objectif :** Garantir la robustesse via une couverture minimale de 80%.

7.1. Stratégie et Résultats

- **Tests Unitaires** : JUnit 5 + Mockito pour isoler la logique métier (notamment les Schedulers).
- **Tests d'Intégration** : MockMvc + base H2 pour valider les APIs et le contrôle d'accès (RBAC).
- **Couverture JaCoCo** : Supérieure à 80% sur l'ensemble du périmètre métier des deux services.

8. Déploiement Kubernetes et Helm

8.1. Isolation de l'infrastructure (Helm)

- **Chart fleet-infra** : Déploiement de bases PostgreSQL isolées (`postgres-maintenance` et `postgres-driver`).
- **Initialisation** : Utilisation de ConfigMaps pour injecter les schémas SQL spécifiques au démarrage.